

C# BACKEND TRACK | CODELINE BOOTCAMP

Session Material

Web API Development

Project Setup · Dependency Injection · Middleware & Pipeline
Connection String · Controllers · How Requests Are Accepted
Parameter Binding · Validation · DTOs · CORS

All examples are from the FirstWebApp project — Product model, Repo, Service, Controller

WHAT WE COVER IN THIS MATERIAL

01	Project Setup	Project structure, launchSettings.json, appsettings.json, Program.cs — two areas
02	Dependency Injection	Service container, AddScoped, constructor injection through Repo → Service → Controller
03	Middleware & Request Pipeline	What middleware is, pipeline order, built-in middleware explained
04	Connection String & Configuration	appsettings.json, GetConnectionString, AddDbContext registration
05	Controllers	ControllerBase, [ApiController], [Route], HTTP verbs, IActionResult, status codes
06	How Controllers Accept Requests	Request lifecycle — how a URL + verb reaches an action method (routing)
07	Parameter Binding	[FromBody], [FromQuery], [FromRoute], [FromHeader] — with Product examples
08	Validation	Model annotations, automatic 400, ModelState, annotating DTOs
09	DTOs — Data Transfer Objects	Why DTOs, Request vs Response DTO, manual mapping in service layer
10	CORS	What CORS is, AddCors, policy, UseCors in the middleware pipeline

01 PROJECT SETUP

An ASP.NET Core Web API project is a .NET application that uses the **Microsoft.NET.Sdk.Web** SDK. This gives it a built-in HTTP web server (Kestrel), a routing engine, a middleware pipeline, and the ability to respond to HTTP requests — none of which exist in a console project. Three files control how the application starts and how it behaves:

launchSettings.json	Lives in the Properties/ folder. Used only by Visual Studio during development — never deployed. Controls which URL(s) the app listens on (HTTP + HTTPS ports), which environment name is active (Development / Production), and whether the browser opens on startup. Different launch profiles let you run the app under different settings without changing any code.
appsettings.json	Stores application-level configuration that can differ between environments. Typical contents: database connection strings, file paths, encryption keys, email sender addresses, API keys, feature flags. A companion file appsettings.Development.json overrides values for the Development environment only — the framework merges them automatically. Values are injected into services through builder.Configuration .
Program.cs	The entry point of the application. It has two clearly separated areas: (1) the Service Container — everything between WebApplication.CreateBuilder() and builder.Build() . This is where you register all dependencies, the DbContext, controllers, Swagger, CORS. (2) the Middleware Pipeline — everything after builder.Build() . This is where you define in what order each incoming HTTP request is processed before reaching a controller.

Program.cs — FirstWebApp

```
// Program.cs - FirstWebApp (two areas clearly marked)

var builder = WebApplication.CreateBuilder(args);

// ■■ AREA 1: SERVICE CONTAINER ████████████████████████████████████

builder.Services.AddDbContext<ProjectContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString(DefaultConnection)));

builder.Services.AddScoped<ProductRepo>();
builder.Services.AddScoped<ProductService>();
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// ■■ AREA 2: MIDDLEWARE PIPELINE ████████████████████████████████████

app.UseSwagger();
app.UseSwaggerUI();
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

02 DEPENDENCY INJECTION

Dependency Injection (DI) is the framework doing the manual wiring for you. In the console SoC project we wrote **ProductRepo repo = new ProductRepo();** inside the service. With DI you register a class once in the service container and the framework creates and provides it automatically wherever it is needed — through constructors.

How DI flows in the FirstWebApp project:

Program.cs — registration

```
// STEP 1 – Register in service container (Program.cs)
builder.Services.AddScoped<ProductRepo>();
builder.Services.AddScoped<ProductService>();
// framework will now create and inject these wherever a constructor requests them
```

Repositories/ProductRepo.cs

```
// STEP 2 – ProductRepo receives ProjectContext from the framework
public class ProductRepo
{
    // ProjectContext context = new ProjectContext(); ← old manual creation
    private ProjectContext context;
    public ProductRepo(ProjectContext _context)
    {
        context = _context; // framework provides this
    }
}
```

Services/ProductService.cs

```
// STEP 3 – ProductService receives ProductRepo from the framework
public class ProductService
{
    // ProductRepo repo = new ProductRepo(); ← old manual creation
    private ProductRepo repo;
    public ProductService(ProductRepo _repo)
    {
        repo = _repo;
    }
}
```

Controllers/ProductController.cs

```
// STEP 4 – ProductController receives ProductService from the framework
public class ProductController : ControllerBase
{
    // ProductService productService = new ProductService(); ← old manual creation
    private ProductService productService;
    public ProductController(ProductService _productService)
    {
        productService = _productService;
    }
}
```

Service lifetimes — AddScoped vs AddSingleton vs AddTransient:

Method	One instance per...	Use for
AddScoped	HTTP request	Repositories, Services, DbContext — most common choice
AddSingleton	Application lifetime	Config readers, caches, thread-safe stateless utilities
AddTransient	Every injection	Lightweight stateless helpers with no shared state

03 MIDDLEWARE & REQUEST PIPELINE

Every HTTP request that arrives at the API travels through a **chain of middleware components** before reaching a controller, and back through the same chain on the way out. Each middleware can inspect or modify the request, do its own work, and then pass control to the next middleware — or short-circuit and return a response immediately without going further.

Order is critical. Middleware runs in the exact sequence you register it with `app.UseXxx()`. If you put `UseAuthorization()` before `UseAuthentication()`, the framework tries to authorize a user it has not yet identified. If you put `MapControllers()` before `UseAuthorization()`, controllers respond without any auth check.

Program.cs — middleware pipeline

```
// Correct pipeline order – from FirstWebApp Program.cs
app.UseSwagger(); // serves the OpenAPI JSON spec
app.UseSwaggerUI(); // interactive UI at /swagger
app.UseHttpsRedirection(); // redirects HTTP to HTTPS
app.UseCors(); // cross-origin headers ← covered in topic 10
app.UseAuthentication(); // who is the user? ← future session
app.UseAuthorization(); // does the user have permission?
app.MapControllers(); // route to the right controller action
app.Run();
```

Middleware	What it does
<code>UseSwagger</code> / <code>UseSwaggerUI</code>	Serves interactive API docs at /swagger — enabled in Development only
<code>UseHttpsRedirection</code>	Redirects plain HTTP requests to their HTTPS equivalent automatically
<code>UseCors</code>	Adds CORS headers so browsers allow cross-origin requests from the frontend
<code>UseAuthentication</code>	Reads the JWT token / cookie and sets the current user identity
<code>UseAuthorization</code>	Checks [Authorize] attributes — returns 401 or 403 if not allowed
<code>MapControllers</code>	Routes the request to the matching controller action method

04 CONNECTION STRING & CONFIGURATION

In the console app, the connection string was hardcoded inside `OnConfiguring()` in the context class. In the Web API project the connection string moves to `appsettings.json` and the context is registered through DI with a constructor that receives `DbContextOptions`.

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=ProjectDataStore;
    Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

ProjectContext.cs — from FirstWebApp

```
// ProjectContext.cs – constructor injection replaces OnConfiguring()
public class ProjectContext : DbContext
{
  public DbSet<Product> Products { get; set; }
  // DbContextOptions is injected by the framework
  public ProjectContext(DbContextOptions<ProjectContext> options) : base(options)
  {
  }
}
```

Program.cs — AddDbContext

```
// Program.cs – connect the context to appsettings.json
builder.Services.AddDbContext<ProjectContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString(DefaultConnection)
)
);
```

Why move it out of OnConfiguring? When the string is hardcoded in the context you cannot change it per environment. With `appsettings` you can have `appsettings.Development.json` pointing to localhost and `appsettings.Production.json` pointing to the cloud server — the framework picks the right file automatically based on the environment name.

05 CONTROLLERS

A controller is a class that receives HTTP requests and returns HTTP responses. It replaces the Presentation layer from the SoC console project. Instead of `Console.ReadLine()` it reads from the HTTP request. Instead of `Console.WriteLine()` it returns `Ok()`, `NotFound()`, etc.

Four essential parts of a controller — from `ProductController.cs`:

Controllers/ProductController.cs

```
// 1. [ApiController] – marks this class as a Web API controller
// enables automatic model validation, automatic [FromBody] binding
[ApiController]

// 2. [Route] – base URL for all actions in this controller
// ProductController → route base = /product
[Route("product")]

public class ProductController : ControllerBase
{
    // 3. Constructor injection – framework provides ProductService
    private ProductService productService;
    public ProductController(ProductService _productService)
    { productService = _productService; }

    // 4. Action methods – each maps to one HTTP endpoint
    [HttpGet("GetAllProducts")]
    public IActionResult GetAllProducts()
    {
        return Ok(productService.GetAllProducts());
    }

    [HttpDelete("Delete")]
    public IActionResult Delete(int productId)
    {
        bool deleted = productService.Delete(productId);
        if (!deleted) return NotFound();
        return NoContent();
    }
}
```

IActionResult helpers and their HTTP status codes:

Method	Status	When to use
<code>Ok(data)</code>	200	Successful GET or PUT — returns data in the response body
<code>Created(url, data)</code>	201	Successful POST — new resource was created
<code>CreatedAtAction(..)</code>	201	POST — includes the URL of the new resource in the Location header

Method	Status	When to use
<code>NoContent()</code>	204	Successful DELETE or update with no body to return
<code>BadRequest(obj)</code>	400	Client sent invalid data or a business rule was violated
<code>NotFound(obj)</code>	404	The requested resource does not exist
<code>Unauthorized()</code>	401	Not authenticated — no valid token provided (future session)

06 HOW CONTROLLERS ACCEPT REQUESTS

When a request arrives at the API, the routing engine reads the **HTTP method** (GET, POST, PUT, DELETE) and the **URL path**, then finds the exact controller class and action method that matches both. This matching process is called **routing**.

Step-by-step: how GET /product/GetProductById?id=3 reaches the action:

Controllers/ProductController.cs — full route map

```
// Request arrives: GET /product/GetProductById?id=3
// Step 1: routing engine reads HTTP method → GET
// Step 2: routing engine reads URL path → /product/GetProductById
// Step 3: matches [Route("product")] on ProductController
// Step 4: matches [HttpGet("GetProductById")] on the GetProductById() method
// Step 5: reads ?id=3 from query string and binds it to the int id parameter
// Step 6: calls GetProductById(3), returns Ok(product)

[ApiController]
[Route("product")] ← base of all routes in this controller
public class ProductController : ControllerBase
{
    [HttpGet(GetAllProducts)] ← GET /product/GetAllProducts
    public IActionResult GetAllProducts() { ... }

    [HttpGet(GetProductById)] ← GET /product/GetProductById?id=3
    public IActionResult GetProductById(int id) { ... }

    [HttpPost(Add)] ← POST /product/Add
    public IActionResult Add(Product product) { ... }

    [HttpPut(UpdatePrice)] ← PUT /product/UpdatePrice
    public IActionResult UpdatePrice(int productId, int newPrice) { ... }

    [HttpDelete(Delete)] ← DELETE /product/Delete
    public IActionResult Delete(int productId) { ... }
}
```

The complete endpoint table for the Product API:

Verb	URL	Action	Returns
GET	/product/GetAllProducts	GetAllProducts()	200 + list of products
GET	/product/GetProductById?id=3	GetProductById(id)	200 + product or nothing
POST	/product/Add	Add(product)	200 + { ProductId }
PUT	/product/UpdatePrice	UpdatePrice(id, price)	204 or 404
DELETE	/product/Delete	Delete(productId)	204 or 404

07 PARAMETER BINDING

Parameter binding is how ASP.NET Core reads values from the incoming HTTP request and maps them to action method parameters. There are four sources — you tell the framework which one to use with an attribute. When **[ApiController]** is present, simple types (int, string, bool) default to **[FromQuery]** and complex types (classes) default to **[FromBody]**.

Attribute	Source	Example
[FromQuery]	URL query string (?key=value)	GET /product/GetProductById?id=3
[FromRoute]	URL path segment {id}	GET /product/{id} ← id embedded in URL
[FromBody]	JSON request body	POST /product/Add body: { Name, Description, Price }
[FromHeader]	HTTP request header	Authorization: Bearer token123

Controllers/ProductController.cs — all binding attributes shown

```
// [FromQuery] – id comes from the query string: ?id=3
[HttpGet(GetProductId)]
public IActionResult GetProductId([FromQuery] int id)
{
    Product product = productService.GetProductId(id);
    return Ok(product);
}

// [FromBody] – product object comes from the JSON request body
[HttpPost(Add)]
public IActionResult Add([FromBody] Product product)
{
    int productId = productService.Create(product);
    return Ok(new { ProductId = productId });
}

// Multiple [FromQuery] – both come from the query string
// PUT /product/UpdatePrice?productId=3&newPrice=999
[HttpPut(UpdatePrice)]
public IActionResult UpdatePrice([FromQuery] int productId, [FromQuery] int newPrice)
{
    bool updated = productService.UpdatePrice(productId, newPrice);
    if (!updated) return NotFound();
    return NoContent();
}

// [FromRoute] – id embedded in the URL path
// DELETE /product/5
[HttpDelete({id})]
public IActionResult Delete([FromRoute] int id)
{
    bool deleted = productService.Delete(id);
    if (!deleted) return NotFound();
    return NoContent();
}
```

08 VALIDATION

Validation ensures that data arriving from the client is correct before you try to use it. ASP.NET Core with **[ApiController]** handles this automatically using the annotations on your model or DTO class — no extra code inside the action method is needed.

The Product model already has annotations — they drive validation automatically:

Models/Product.cs

```
// Models/Product.cs — from FirstWebApp
public class Product
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    [Required] ← cannot be null or empty — [ApiController] returns 400 if violated
    public string Name { get; set; }
    [Required]
    public string Description { get; set; }
    [Required]
    public int Price { get; set; }
}
```

When **[ApiController]** is on the controller, the framework checks the incoming object against all annotations before your action method runs. If any rule fails it immediately returns a **400 Bad Request** with a structured error body — your code never executes:

Automatic 400 response — [ApiController] does the work

```
// Client sends: POST /product/Add
{
    "Name": "", ← empty — [Required] fails
    "Description": null, ← null — [Required] fails
    "Price": 0
}

// Framework automatically returns 400:
{
    "status": 400,
    "errors": {
        "Name": [ "The Name field is required." ],
        "Description": [ "The Description field is required." ]
    }
}

// Your Add() action method never runs — no if-statement needed
```

Common validation annotations and what they enforce:

Annotation	Rule enforced
<code>[Required]</code>	Field cannot be null or empty
<code>[MaxLength(n)]</code>	String cannot exceed n characters
<code>[MinLength(n)]</code>	String must have at least n characters
<code>[Range(min, max)]</code>	Number must be within the given range
<code>[EmailAddress]</code>	Must be a valid email format (has @ and domain)
<code>[RegularExpression(p)]</code>	Must match the given regex pattern
<code>[Compare(nameof(F))]</code>	Must equal another field (e.g. <code>ConfirmPassword == Password</code>)

Model validation vs business validation: `[Required]` and `[Range]` handle format/structure rules automatically. Business rules — like 'a product with this name already exists' or 'the categoryId must refer to a real category' — are checked manually inside the service or controller and you return `BadRequest ( )` with a descriptive message yourself.

09 DTOs — DATA TRANSFER OBJECTS

A DTO (Data Transfer Object) is a plain class that defines the exact shape of data travelling into or out of an endpoint. The **model** is for the database. The **DTO** is for the HTTP layer. You never expose a raw model class directly through an API.

Returning the Product model directly — problems:

- Exposes database column names and internal IDs directly
- Client could attempt to send and overwrite Id or other system fields
- Navigation properties cause circular JSON serialization errors
- A database schema change instantly breaks the API response shape
- No way to add computed or joined fields to the response

Using a DTO — what it solves:

- You control exactly which fields the client sees and can send
- Request DTO defines only what the client is allowed to set
- No navigation properties — no circular reference risk
- API shape stays stable even if the model changes internally
- Easy to add computed fields (e.g. CategoryName from a join)

Two DTOs for the Product — Request and Response:

DTOs/CreateProductDto.cs & DTOs/ProductResponseDto.cs

```
// DTOs/CreateProductDto.cs — what the client sends on POST
// Client ONLY provides fields they should control
public class CreateProductDto
{
    [Required(ErrorMessage = Name is required.)]
    [MaxLength(150)]
    public string Name { get; set; }

    [Required(ErrorMessage = Description is required.)]
    public string Description { get; set; }

    [Required(ErrorMessage = Price is required.)]
    [Range(1, 999999, ErrorMessage = Price must be greater than 0.)]
    public int Price { get; set; }

    // NO Id — the database generates it automatically
}

// DTOs/ProductResponseDto.cs — what the API sends back
public class ProductResponseDto
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public int Price { get; set; }

    // no navigation properties — safe to serialize, no circular reference
}
```

Manual mapping — from DTO to model and back (in ProductService.cs):

Services/ProductService.cs + Controllers/ProductController.cs

```
// Services/ProductService.cs – updated to accept DTO and return DTO
public ProductResponseDto Create(CreateProductDto dto)
{
    // Map DTO → Model before saving to the database
    Product product = new Product();
    product.Name = dto.Name;
    product.Description = dto.Description;
    product.Price = dto.Price;
    repo.Add(product);

    // Map Model → Response DTO before returning to the controller
    ProductResponseDto response = new ProductResponseDto();
    response.Id = product.Id;
    response.Name = product.Name;
    response.Description = product.Description;
    response.Price = product.Price;
    return response;
}

// Controllers/ProductController.cs – receives DTO, returns DTO
[HttpPost(Add)]
public IActionResult Add([FromBody] CreateProductDto dto)
{
    ProductResponseDto created = productService.Create(dto);
    return CreatedAtAction(nameof(GetProductById), new { id = created.Id }, created);
}
```


Method	What it allows
<code>AllowCredentials()</code>	Allows cookies and Authorization headers — cannot combine with <code>AllowAnyOrigin</code>

The complete Program.cs with all 10 topics applied:

Program.cs — complete with all 10 topics applied

```
var builder = WebApplication.CreateBuilder(args);  
  
// ■■ SERVICE CONTAINER  
// 04 – DbContext + connection string from appsettings.json  
builder.Services.AddDbContext<ProjectContext>(options =>  
options.UseSqlServer(  
builder.Configuration.GetConnectionString(DefaultConnection)));  
  
// 02 – Dependency Injection: register repo and service  
builder.Services.AddScoped<ProductRepo>();  
builder.Services.AddScoped<ProductService>();  
  
// 10 – CORS: allow the Angular frontend  
builder.Services.AddCors(o => o.AddPolicy(AllowFrontend,  
p => p.WithOrigins(http://localhost:4200).AllowAnyHeader().AllowAnyMethod()));  
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
  
var app = builder.Build();  
  
// ■■ MIDDLEWARE PIPELINE (order matters)  
app.UseSwagger();  
app.UseSwaggerUI();  
app.UseHttpsRedirection();  
app.UseCors(AllowFrontend); ← must be before UseAuthorization  
app.UseAuthorization();  
app.MapControllers();  
app.Run();
```